Message sprint 1 Database socket

MESSAGING SPRINT 1 — DATABASE + SOCKET.IO + MESSAGE CRUD

Build ALL schemas + real-time engine + basic message operations

PASTE THIS INTO REPLIT AI AGENT

WHAT TO BUILD

1. All Prisma models for the messaging system
2. Socket.IO server with Redis adapter
3. Presence service (online/offline/away/DND tracking)
4. Typing indicator service
5. Unread count tracking service
6. Message CRUD tRPC router (send, edit, delete, list, pin, react)

Verify after this sprint:

- Socket.IO server connects and pings
- Send a message via tRPC → appears in database
- Presence: user goes online → other users see green dot
- Typing: user types → other users see "Alice is typing..."

STEP 1: PRISMA SCHEMA

Add these models to your existing `prisma/schema.prisma`:

```
// ============================================================
==
// MESSAGING SYSTEM — ALL MODELS
// ============================================================
==

model Channel {
  id                String    @id @default(uuid()) @db.Uuid
  workspaceId       String    @db.Uuid
  name              String    @db.VarChar(80) // lowercase, h
yphens, no spaces
  displayName       String    @db.VarChar(80) // human-friend
ly
  slug              String    // URL-safe
  description       String?   @db.VarChar(500)
  topic             String?   @db.VarChar(250) // shown in he
ader
  type              String    @default("public") // 'publi
c','private','project','team','announcement','social'
  icon              String?   // emoji or custom icon URL
  color             String?   // hex color for sidebar
  isDefault         Boolean   @default(false) // auto-join fo
r new members
  isArchived        Boolean   @default(false)
  isPinned          Boolean   @default(false) // workspace-le
vel pin by admins
  linkedProjectId   String?   @db.Uuid
  linkedTeamId      String?   @db.Uuid
  postingPermission String    @default("everyone") // 'everyo
ne','admins_only','designated_posters'
  memberCount       Int       @default(0)
  messageCount      Int       @default(0)
  lastMessageAt     DateTime?
  lastMessagePreview String?  @db.VarChar(200)
```

```
  retentionDays     Int?      // channel-specific retention o
verride
  createdBy         String    @db.Uuid
  createdAt         DateTime  @default(now())
  updatedAt         DateTime  @updatedAt
  deletedAt         DateTime?

  members           ChannelMember[]
  messages          Message[]
  pins              PinnedMessage[]

  @@unique([workspaceId, name])
  @@index([workspaceId, type])
  @@index([workspaceId, lastMessageAt(sort: Desc)])
  @@index([linkedProjectId])
}

model ChannelMember {
  id                 String    @id @default(uuid()) @db.Uuid
  channelId          String    @db.Uuid
  userId             String    @db.Uuid
  role               String    @default("member") // 'owne
r','admin','poster','member','guest'
  joinedAt           DateTime  @default(now())
  lastReadAt         DateTime  @default(now())
  lastReadMessageId  String?   @db.Uuid
  notificationPref   String    @default("all") // 'all','ment
ions_only','nothing'
  isMuted            Boolean   @default(false)
  isStarred          Boolean   @default(false)
  sidebarSection     String?   // custom sidebar section grou
ping

  channel            Channel   @relation(fields: [channelId],
references: [id], onDelete: Cascade)
```

```
  @@unique([channelId, userId])
  @@index([userId, isStarred])
  @@index([userId, lastReadAt])
}

model Message {
  id              String    @id @default(uuid()) @db.Uuid
  channelId       String    @db.Uuid
  authorId        String    @db.Uuid
  content         String    @db.Text // plain text or markdow
n
  contentHtml     String?   @db.Text // rendered HTML (from r
ich editor)
  contentBlocks   Json?     // structured blocks [{type:'para
graph'|'code'|'quote'|'heading'|'list',content:string,languag
e?:string}]
  type            String    @default("user") // 'user','syste
m','bot','ai','scheduled'
  threadParentId  String?   @db.Uuid // if this is a thread r
eply
  threadReplyCount Int      @default(0) // denormalized count
of replies
  threadLastReplyAt DateTime? // for sorting threads
  threadParticipantIds String[] @db.Uuid // users who replied
in thread
  isEdited        Boolean   @default(false)
  editedAt        DateTime?
  isPinned        Boolean   @default(false)
  pinnedAt        DateTime?
  pinnedBy        String?   @db.Uuid
  isBookmarked    Boolean   @default(false)
  scheduledFor    DateTime? // if type='scheduled', when to d
eliver
  metadata        Json?     // link previews, bot data, speci
al formatting
  mentionedUserIds String[] @db.Uuid // for notification targ
```

```
eting
  mentionedChannelIds String[] @db.Uuid
  hasAttachments  Boolean   @default(false)
  reactionSummary Json?     // {emoji: count, emoji: count} d
enormalized for fast display
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt
  deletedAt       DateTime? // soft delete: shows "This messa
ge was deleted"

  channel         Channel   @relation(fields: [channelId], re
ferences: [id])
  attachments     MessageAttachment[]
  reactions       MessageReaction[]

  @@index([channelId, createdAt(sort: Desc)])
  @@index([channelId, threadParentId])
  @@index([authorId, createdAt(sort: Desc)])
  @@index([channelId, isPinned])
}

model MessageReaction {
  id        String   @id @default(uuid()) @db.Uuid
  messageId String   @db.Uuid
  userId    String   @db.Uuid
  emoji     String   // ":thumbsup:" or unicode "👍"
  createdAt DateTime @default(now())

  message   Message  @relation(fields: [messageId], reference
s: [id], onDelete: Cascade)

  @@unique([messageId, userId, emoji]) // one reaction per em
oji per user per message
  @@index([messageId])
}
```

```
model MessageAttachment {
  id          String   @id @default(uuid()) @db.Uuid
  messageId   String   @db.Uuid
  fileName    String
  fileUrl     String   // S3/R2 URL
  fileSize    Int      // bytes
  mimeType    String
  thumbnailUrl String? // for images/videos/PDFs
  width       Int?     // for images/videos
  height      Int?     // for images/videos
  duration    Int?     // for audio/video (seconds)
  createdAt   DateTime @default(now())

  message     Message  @relation(fields: [messageId], referen
ces: [id], onDelete: Cascade)

  @@index([messageId])
}

model PinnedMessage {
  id        String   @id @default(uuid()) @db.Uuid
  channelId String   @db.Uuid
  messageId String   @db.Uuid
  pinnedBy  String   @db.Uuid
  pinnedAt  DateTime @default(now())

  channel   Channel  @relation(fields: [channelId], reference
s: [id], onDelete: Cascade)

  @@unique([channelId, messageId])
  @@index([channelId, pinnedAt(sort: Desc)])
}

model Conversation {
  id              String    @id @default(uuid()) @db.Uuid
  workspaceId     String    @db.Uuid
```

```
  type            String    @default("dm") // 'dm' (1:1) or
'group'
  name            String?   // custom group name (null for 1:
1)
  icon            String?   // custom group icon
  participantIds  String[]  @db.Uuid // ordered, used for ded
up
  participantHash String    // hash of sorted participantIds
for fast dedup lookup
  lastMessageAt   DateTime?
  lastMessagePreview String? @db.VarChar(200)
  messageCount    Int       @default(0)
  createdBy       String    @db.Uuid
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt

  members         ConversationMember[]
  dmMessages      DirectMessage[]

  @@unique([workspaceId, participantHash]) // prevent duplica
te conversations
  @@index([workspaceId])
}

model ConversationMember {
  id               String   @id @default(uuid()) @db.Uuid
  conversationId   String   @db.Uuid
  userId           String   @db.Uuid
  lastReadAt       DateTime @default(now())
  lastReadMessageId String? @db.Uuid
  isMuted          Boolean  @default(false)
  joinedAt         DateTime @default(now())

  conversation     Conversation @relation(fields: [conversati
onId], references: [id], onDelete: Cascade)
```

```
  @@unique([conversationId, userId])
  @@index([userId])
}

model DirectMessage {
  id              String    @id @default(uuid()) @db.Uuid
  conversationId  String    @db.Uuid
  authorId        String    @db.Uuid
  content         String    @db.Text
  contentHtml     String?   @db.Text
  type            String    @default("user") // 'user','syste
m'
  threadParentId  String?   @db.Uuid
  threadReplyCount Int      @default(0)
  isEdited        Boolean   @default(false)
  editedAt        DateTime?
  mentionedUserIds String[] @db.Uuid
  hasAttachments  Boolean   @default(false)
  reactionSummary Json?
  metadata        Json?
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt
  deletedAt       DateTime?

  conversation    Conversation @relation(fields: [conversatio
nId], references: [id])
  attachments     DmAttachment[]
  reactions       DmReaction[]

  @@index([conversationId, createdAt(sort: Desc)])
}

model DmAttachment {
  id          String   @id @default(uuid()) @db.Uuid
  messageId   String   @db.Uuid
  fileName    String
```

```
  fileUrl     String
  fileSize    Int
  mimeType    String
  thumbnailUrl String?
  createdAt   DateTime @default(now())

  message     DirectMessage @relation(fields: [messageId], re
ferences: [id], onDelete: Cascade)
  @@index([messageId])
}

model DmReaction {
  id        String   @id @default(uuid()) @db.Uuid
  messageId String   @db.Uuid
  userId    String   @db.Uuid
  emoji     String
  createdAt DateTime @default(now())

  message   DirectMessage @relation(fields: [messageId], refe
rences: [id], onDelete: Cascade)
  @@unique([messageId, userId, emoji])
}

model CustomEmoji {
  id          String   @id @default(uuid()) @db.Uuid
  workspaceId String   @db.Uuid
  name        String   // :company-logo:
  imageUrl    String
  createdBy   String   @db.Uuid
  createdAt   DateTime @default(now())

  @@unique([workspaceId, name])
}

model UserPresence {
  id          String    @id @default(uuid()) @db.Uuid
```

```
  userId      String    @unique @db.Uuid
  workspaceId String    @db.Uuid
  status      String    @default("offline") // 'online','awa
y','dnd','offline','focusing','in_meeting'
  customText  String?   @db.VarChar(100) // "Out for lunch un
til 2pm"
  customEmoji String?
  lastSeenAt  DateTime  @default(now())
  autoStatus  Boolean   @default(true) // auto-set based on a
ctivity
  dndUntil    DateTime? // auto-clear DND after this time
  updatedAt   DateTime  @updatedAt
}

model Huddle {
  id            String    @id @default(uuid()) @db.Uuid
  channelId     String    @db.Uuid
  workspaceId   String    @db.Uuid
  startedBy     String    @db.Uuid
  status        String    @default("active") // 'active','end
ed'
  participantIds String[] @db.Uuid
  maxParticipants Int     @default(50)
  isRecording   Boolean   @default(false)
  recordingUrl  String?
  liveCaption   Boolean   @default(true)
  transcript    String?   @db.Text
  aiSummary     String?   @db.Text
  startedAt     DateTime  @default(now())
  endedAt       DateTime?

  @@index([channelId, status])
}

model MessageBookmark {
  id        String   @id @default(uuid()) @db.Uuid
```

```
  userId    String   @db.Uuid
  messageId String   @db.Uuid
  messageType String @default("channel") // 'channel' or 'dm'
  note      String?  @db.VarChar(200)
  createdAt DateTime @default(now())

  @@unique([userId, messageId])
  @@index([userId, createdAt(sort: Desc)])
}

model SlashCommand {
  id          String  @id @default(uuid()) @db.Uuid
  workspaceId String? @db.Uuid // null = built-in
  name        String  // /remind, /poll, /task
  description String
  handler     String  // function name or webhook URL
  isBuiltIn   Boolean @default(false)
  createdBy   String? @db.Uuid
  createdAt   DateTime @default(now())

  @@unique([workspaceId, name])
}
```

Run: `npx prisma migrate dev --name add_messaging_system`

STEP 2: SOCKET.IO SERVER

Create file: `src/server/services/socket-server.ts`

```
// src/server/services/socket-server.ts
// Socket.IO server setup with Redis adapter for multi-server
scaling

import { Server as SocketIOServer } from "socket.io";
import { createAdapter } from "@socket.io/redis-adapter";
import { createClient } from "redis";
```

```
import { getServerSession } from "next-auth";
import { authOptions } from "@/lib/auth";
import { registerSocketHandlers } from "./socket-handlers";

let io: SocketIOServer | null = null;

export function getIO(): SocketIOServer {
  if (!io) throw new Error("Socket.IO not initialized");
  return io;
}

export async function initSocketServer(httpServer: any): Prom
ise<SocketIOServer> {
  if (io) return io;

  io = new SocketIOServer(httpServer, {
    path: "/api/socket",
    cors: { origin: process.env.NEXT_PUBLIC_APP_URL, credenti
als: true },
    transports: ["websocket", "polling"],
    pingInterval: 25000,
    pingTimeout: 20000,
  });

  // Redis adapter for horizontal scaling
  if (process.env.REDIS_URL) {
    const pubClient = createClient({ url: process.env.REDIS_U
RL });
    const subClient = pubClient.duplicate();
    await Promise.all([pubClient.connect(), subClient.connect
()]);
    io.adapter(createAdapter(pubClient, subClient));
  }

  // Auth middleware — verify user on every connection
  io.use(async (socket, next) => {
```

```
    try {
      const session = await getServerSession(authOptions);
      if (!session?.user?.id) {
        return next(new Error("Unauthorized"));
      }
      socket.data.userId = session.user.id;
      socket.data.userName = session.user.name;
      next();
    } catch (err) {
      next(new Error("Authentication failed"));
    }
  });

  // Register all event handlers
  io.on("connection", (socket) => {
    console.log(`User connected: ${socket.data.userId}`);
    registerSocketHandlers(io!, socket);
  });

  return io;
}
```

Create file: `src/server/services/socket-handlers.ts`

```
// src/server/services/socket-handlers.ts
// All Socket.IO event handlers

import { Server as SocketIOServer, Socket } from "socket.io";
import { prisma } from "@/lib/prisma";
import { presenceService } from "./presence-service";
import { typingService } from "./typing-service";
import { unreadService } from "./unread-service";

export function registerSocketHandlers(io: SocketIOServer, so
cket: Socket) {
  const userId = socket.data.userId;
```

```
  const userName = socket.data.userName;

  // ==========================================================
====
  // CONNECTION / DISCONNECTION
  // ==========================================================
====

  // Join user's workspace rooms
  socket.on("join_workspace", async (data: { workspaceId: str
ing }) => {
    socket.join(`workspace:${data.workspaceId}`);

    // Set user online
    await presenceService.setOnline(userId, data.workspaceI
d);

    // Broadcast presence to workspace
    io.to(`workspace:${data.workspaceId}`).emit("presence_upd
ate", {
      userId,
      status: "online",
      lastSeenAt: new Date().toISOString(),
    });

    // Join all channels the user is a member of
    const memberships = await prisma.channelMember.findMany({
      where: { userId },
      select: { channelId: true },
    });
    for (const m of memberships) {
      socket.join(`channel:${m.channelId}`);
    }

    // Join all DM conversations
    const conversations = await prisma.conversationMember.fin
```

```
dMany({
      where: { userId },
      select: { conversationId: true },
    });
    for (const c of conversations) {
      socket.join(`conversation:${c.conversationId}`);
    }
  });

  socket.on("disconnect", async () => {
    console.log(`User disconnected: ${userId}`);
    // Don't set offline immediately — wait 30s for reconnect
    setTimeout(async () => {
      const currentSocket = await io.in(`user:${userId}`).fet
chSockets();
      if (currentSocket.length === 0) {
        const presence = await prisma.userPresence.findUnique
({ where: { userId } });
        if (presence) {
          await presenceService.setOffline(userId, presence.w
orkspaceId);
          io.to(`workspace:${presence.workspaceId}`).emit("pr
esence_update", {
            userId,
            status: "offline",
            lastSeenAt: new Date().toISOString(),
          });
        }
      }
    }, 30000);
  });

  // ==========================================================
====
  // MESSAGING
  // ==========================================================
```

```
====

  // New message in channel
  socket.on("message_send", async (data: {
    channelId: string;
    content: string;
    contentHtml?: string;
    threadParentId?: string;
    mentionedUserIds?: string[];
    attachmentIds?: string[];
    tempId: string; // client-side temp ID for optimistic UI
  }) => {
    try {
      // Permission check
      const membership = await prisma.channelMember.findUniqu
e({
        where: { channelId_userId: { channelId: data.channelI
d, userId } },
      });
      if (!membership) {
        socket.emit("message_error", { tempId: data.tempId, e
rror: "Not a member of this channel" });
        return;
      }

      // Check posting permission
      const channel = await prisma.channel.findUnique({ wher
e: { id: data.channelId } });
      if (channel?.postingPermission === "admins_only" && !
["owner", "admin"].includes(membership.role)) {
        socket.emit("message_error", { tempId: data.tempId, e
rror: "Only admins can post in this channel" });
        return;
      }

      // Create message
```

```
      const message = await prisma.message.create({
        data: {
          channelId: data.channelId,
          authorId: userId,
          content: data.content,
          contentHtml: data.contentHtml,
          threadParentId: data.threadParentId,
          mentionedUserIds: data.mentionedUserIds || [],
          hasAttachments: (data.attachmentIds || []).length >
0,
        },
        include: {
          attachments: true,
        },
      });

      // Update channel stats
      await prisma.channel.update({
        where: { id: data.channelId },
        data: {
          messageCount: { increment: 1 },
          lastMessageAt: new Date(),
          lastMessagePreview: data.content.slice(0, 200),
        },
      });

      // If thread reply, update parent
      if (data.threadParentId) {
        await prisma.message.update({
          where: { id: data.threadParentId },
          data: {
            threadReplyCount: { increment: 1 },
            threadLastReplyAt: new Date(),
            threadParticipantIds: {
              push: userId,
            },
```

```
          },
        });
      }

      // Broadcast to channel
      io.to(`channel:${data.channelId}`).emit("message_new",
{
        ...message,
        tempId: data.tempId, // so sender can reconcile optim
istic message
        authorName: userName,
      });

      // Update unread counts for all channel members except
sender
      await unreadService.incrementUnread(data.channelId, use
rId);

      // Send notifications for @mentions
      if (data.mentionedUserIds && data.mentionedUserIds.leng
th > 0) {
        for (const mentionedId of data.mentionedUserIds) {
          io.to(`user:${mentionedId}`).emit("notification", {
            type: "mention",
            channelId: data.channelId,
            messageId: message.id,
            authorName: userName,
            preview: data.content.slice(0, 100),
          });
        }
      }

    } catch (error: any) {
      socket.emit("message_error", { tempId: data.tempId, err
or: error.message });
    }
```

```
  });

  // Edit message
  socket.on("message_edit", async (data: { messageId: string;
content: string; contentHtml?: string }) => {
    const message = await prisma.message.findUnique({ where:
{ id: data.messageId } });
    if (!message || message.authorId !== userId) {
      socket.emit("error", { message: "Cannot edit this messa
ge" });
      return;
    }

    const updated = await prisma.message.update({
      where: { id: data.messageId },
      data: { content: data.content, contentHtml: data.conten
tHtml, isEdited: true, editedAt: new Date() },
    });

    io.to(`channel:${message.channelId}`).emit("message_updat
ed", updated);
  });

  // Delete message
  socket.on("message_delete", async (data: { messageId: strin
g }) => {
    const message = await prisma.message.findUnique({ where:
{ id: data.messageId } });
    if (!message) return;

    // Allow delete if author or channel admin
    const membership = await prisma.channelMember.findUnique
({
      where: { channelId_userId: { channelId: message.channel
Id, userId } },
    });
```

```
    if (message.authorId !== userId && ["owner", "admin"].in
cludes(membership?.role || "")) {
      socket.emit("error", { message: "Cannot delete this mes
sage" });
      return;
    }

    await prisma.message.update({
      where: { id: data.messageId },
      data: { deletedAt: new Date() },
    });

    io.to(`channel:${message.channelId}`).emit("message_delet
ed", {
      messageId: data.messageId,
      channelId: message.channelId,
    });
  });

  // Add reaction
  socket.on("reaction_add", async (data: { messageId: string;
emoji: string }) => {
    const message = await prisma.message.findUnique({ where:
{ id: data.messageId } });
    if (!message) return;

    await prisma.messageReaction.upsert({
      where: { messageId_userId_emoji: { messageId: data.mess
ageId, userId, emoji: data.emoji } },
      create: { messageId: data.messageId, userId, emoji: dat
a.emoji },
      update: {},
    });

    // Update denormalized reaction summary
    const reactions = await prisma.messageReaction.groupBy({
```

```
      by: ["emoji"],
      where: { messageId: data.messageId },
      _count: true,
    });
    const summary = Object.fromEntries(reactions.map(r => [r.
emoji, r._count]));
    await prisma.message.update({
      where: { id: data.messageId },
      data: { reactionSummary: summary },
    });

    io.to(`channel:${message.channelId}`).emit("reaction_upda
ted", {
      messageId: data.messageId,
      reactionSummary: summary,
      addedBy: userId,
      emoji: data.emoji,
    });
  });

  // Remove reaction
  socket.on("reaction_remove", async (data: { messageId: stri
ng; emoji: string }) => {
    const message = await prisma.message.findUnique({ where:
{ id: data.messageId } });
    if (!message) return;

    await prisma.messageReaction.deleteMany({
      where: { messageId: data.messageId, userId, emoji: dat
a.emoji },
    });

    const reactions = await prisma.messageReaction.groupBy({
      by: ["emoji"],
      where: { messageId: data.messageId },
      _count: true,
```

```
    });
    const summary = Object.fromEntries(reactions.map(r => [r.
emoji, r._count]));
    await prisma.message.update({
      where: { id: data.messageId },
      data: { reactionSummary: summary },
    });

    io.to(`channel:${message.channelId}`).emit("reaction_upda
ted", {
      messageId: data.messageId,
      reactionSummary: summary,
    });
  });

  // ==========================================================
====
  // TYPING INDICATORS
  // ==========================================================
====

  socket.on("typing_start", async (data: { channelId: string
}) => {
    await typingService.setTyping(userId, userName, data.chan
nelId);
    socket.to(`channel:${data.channelId}`).emit("typing", {
      channelId: data.channelId,
      userId,
      userName,
      isTyping: true,
    });
  });

  socket.on("typing_stop", async (data: { channelId: string
}) => {
    await typingService.clearTyping(userId, data.channelId);
```

```
    socket.to(`channel:${data.channelId}`).emit("typing", {
      channelId: data.channelId,
      userId,
      isTyping: false,
    });
  });

  // ==========================================================
====
  // READ RECEIPTS & MARK READ
  // ==========================================================
====

  socket.on("mark_read", async (data: { channelId: string; me
ssageId: string }) => {
    await prisma.channelMember.update({
      where: { channelId_userId: { channelId: data.channelId,
userId } },
      data: { lastReadAt: new Date(), lastReadMessageId: dat
a.messageId },
    });

    await unreadService.clearUnread(data.channelId, userId);

    // Emit updated unread count to the user
    socket.emit("unread_updated", { channelId: data.channelI
d, unreadCount: 0 });
  });

  // ==========================================================
====
  // PRESENCE
  // ==========================================================
====

  socket.on("presence_set", async (data: { status: string; cu
```

```
stomText?: string }) => {
    const presence = await presenceService.setStatus(userId,
data.status, data.customText);
    const workspaceId = presence?.workspaceId;
    if (workspaceId) {
      io.to(`workspace:${workspaceId}`).emit("presence_updat
e", {
        userId,
        status: data.status,
        customText: data.customText,
        lastSeenAt: new Date().toISOString(),
      });
    }
  });

  // ========================================================
====
  // CHANNEL JOIN/LEAVE
  // ========================================================
====

  socket.on("channel_join", async (data: { channelId: string
}) => {
    socket.join(`channel:${data.channelId}`);
  });

  socket.on("channel_leave", async (data: { channelId: string
}) => {
    socket.leave(`channel:${data.channelId}`);
  });

  // Join user's personal room for DMs and notifications
  socket.join(`user:${userId}`);
}
```

STEP 3: PRESENCE SERVICE

Create file: `src/server/services/presence-service.ts`

```
// src/server/services/presence-service.ts

import { prisma } from "@/lib/prisma";
import { redis } from "@/lib/redis";

const PRESENCE_KEY = "presence:";
const AWAY_TIMEOUT_MS = 5 * 60 * 1000; // 5 minutes

class PresenceService {
  async setOnline(userId: string, workspaceId: string) {
    await redis.setex(`${PRESENCE_KEY}${userId}`, 600, "onlin
e"); // 10 min TTL
    return prisma.userPresence.upsert({
      where: { userId },
      create: { userId, workspaceId, status: "online", lastSe
enAt: new Date() },
      update: { status: "online", lastSeenAt: new Date() },
    });
  }

  async setOffline(userId: string, workspaceId: string) {
    await redis.del(`${PRESENCE_KEY}${userId}`);
    return prisma.userPresence.update({
      where: { userId },
      data: { status: "offline", lastSeenAt: new Date() },
    });
  }

  async setStatus(userId: string, status: string, customTex
t?: string) {
    await redis.setex(`${PRESENCE_KEY}${userId}`, 600, statu
s);
```

```
    return prisma.userPresence.upsert({
      where: { userId },
      create: { userId, workspaceId: "", status, customText,
lastSeenAt: new Date() },
      update: { status, customText, lastSeenAt: new Date() },
    });
  }

  async getStatus(userId: string): Promise<string> {
    return (await redis.get(`${PRESENCE_KEY}${userId}`)) ||
"offline";
  }

  async getWorkspacePresence(workspaceId: string): Promise<Ar
ray<{ userId: string; status: string; customText?: string }>>
{
    const presences = await prisma.userPresence.findMany({
      where: { workspaceId },
      select: { userId: true, status: true, customText: true,
lastSeenAt: true },
    });
    return presences.map(p => ({
      userId: p.userId,
      status: p.status,
      customText: p.customText || undefined,
    }));
  }

  // Heartbeat — called every 30s by client
  async heartbeat(userId: string) {
    await redis.setex(`${PRESENCE_KEY}${userId}`, 600, "onlin
e");
    await prisma.userPresence.update({
      where: { userId },
      data: { lastSeenAt: new Date() },
    }).catch(() => {}); // ignore if not found
```

```
  }
}

export const presenceService = new PresenceService();
```

STEP 4: TYPING SERVICE

Create file: `src/server/services/typing-service.ts`

```
// src/server/services/typing-service.ts

import { redis } from "@/lib/redis";

const TYPING_KEY = "typing:";
const TYPING_TTL = 5; // seconds — auto-expire

class TypingService {
  async setTyping(userId: string, userName: string, channelI
d: string) {
    await redis.setex(`${TYPING_KEY}${channelId}:${userId}`,
TYPING_TTL, userName);
  }

  async clearTyping(userId: string, channelId: string) {
    await redis.del(`${TYPING_KEY}${channelId}:${userId}`);
  }

  async getTypingUsers(channelId: string): Promise<Array<{ us
erId: string; userName: string }>> {
    const keys = await redis.keys(`${TYPING_KEY}${channelId}:
*`);
    const users: Array<{ userId: string; userName: string }>
= [];
    for (const key of keys) {
      const userId = key.split(":").pop()!;
```

```
      const userName = await redis.get(key);
      if (userName) users.push({ userId, userName: userName a
s string });
    }
    return users;
  }
}

export const typingService = new TypingService();
```

STEP 5: UNREAD SERVICE

Create file: `src/server/services/unread-service.ts`

```
// src/server/services/unread-service.ts

import { redis } from "@/lib/redis";

const UNREAD_KEY = "unread:";

class UnreadService {
  // Increment unread for all channel members except the send
er
  async incrementUnread(channelId: string, senderUserId: stri
ng) {
    const { prisma } = await import("@/lib/prisma");
    const members = await prisma.channelMember.findMany({
      where: { channelId, userId: { not: senderUserId } },
      select: { userId: true },
    });

    for (const member of members) {
      await redis.incr(`${UNREAD_KEY}${member.userId}:${chann
elId}`);
    }
```

```
  }

  async clearUnread(channelId: string, userId: string) {
    await redis.del(`${UNREAD_KEY}${userId}:${channelId}`);
  }

  async getUnreadCount(userId: string, channelId: string): Pr
omise<number> {
    const count = await redis.get(`${UNREAD_KEY}${userId}:${c
hannelId}`);
    return parseInt(count as string) || 0;
  }

  async getAllUnreadCounts(userId: string): Promise<Record<st
ring, number>> {
    const keys = await redis.keys(`${UNREAD_KEY}${userId}:*
`);
    const counts: Record<string, number> = {};
    for (const key of keys) {
      const channelId = key.split(":").pop()!;
      const count = await redis.get(key);
      if (count && parseInt(count as string) > 0) {
        counts[channelId] = parseInt(count as string);
      }
    }
    return counts;
  }
}

export const unreadService = new UnreadService();
```

STEP 6: MESSAGE tRPC ROUTER

Create file: `src/server/routers/messages.ts`

```
// src/server/routers/messages.ts
// REST API for messages (Socket.IO handles real-time, tRPC h
andles queries)

import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";

export const messagesRouter = router({
  // Get messages for a channel (paginated, newest first)
  list: protectedProcedure
    .input(z.object({
      channelId: z.string().uuid(),
      limit: z.number().min(1).max(100).default(50),
      cursor: z.string().uuid().optional(), // messageId to p
aginate from
      threadParentId: z.string().uuid().optional(), // for th
read replies
    }))
    .query(async ({ input, ctx }) => {
      // Verify membership
      const membership = await prisma.channelMember.findUniqu
e({
        where: { channelId_userId: { channelId: input.channel
Id, userId: ctx.user.id } },
      });
      if (!membership) throw new Error("Not a member of this
channel");

      const messages = await prisma.message.findMany({
        where: {
          channelId: input.channelId,
          deletedAt: null,
          ...(input.threadParentId
            ? { threadParentId: input.threadParentId }
```

```
            : { threadParentId: null } // only top-level mess
ages in main view
          ),
        },
        orderBy: { createdAt: "desc" },
        take: input.limit + 1,
        ...(input.cursor ? { cursor: { id: input.cursor }, sk
ip: 1 } : {}),
        include: {
          attachments: true,
          reactions: { select: { userId: true, emoji: true }
},
        },
      });

      const hasMore = messages.length > input.limit;
      const items = messages.slice(0, input.limit).reverse();
// reverse to chronological order

      return { messages: items, hasMore, nextCursor: hasMore
? messages[input.limit - 1]?.id : undefined };
    }),

  // Get a single message with thread info
  getById: protectedProcedure
    .input(z.object({ messageId: z.string().uuid() }))
    .query(async ({ input }) => {
      return prisma.message.findUnique({
        where: { id: input.messageId },
        include: { attachments: true },
      });
    }),

  // Pin a message
  pin: protectedProcedure
    .input(z.object({ messageId: z.string().uuid(), channelI
```

```
d: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      await prisma.pinnedMessage.create({
        data: { channelId: input.channelId, messageId: input.
messageId, pinnedBy: ctx.user.id },
      });
      await prisma.message.update({
        where: { id: input.messageId },
        data: { isPinned: true, pinnedAt: new Date(), pinnedB
y: ctx.user.id },
      });
      return { success: true };
    }),

  // Unpin a message
  unpin: protectedProcedure
    .input(z.object({ messageId: z.string().uuid(), channelI
d: z.string().uuid() }))
    .mutation(async ({ input }) => {
      await prisma.pinnedMessage.deleteMany({
        where: { channelId: input.channelId, messageId: inpu
t.messageId },
      });
      await prisma.message.update({
        where: { id: input.messageId },
        data: { isPinned: false, pinnedAt: null, pinnedBy: nu
ll },
      });
      return { success: true };
    }),

  // Get pinned messages for a channel
  getPinned: protectedProcedure
    .input(z.object({ channelId: z.string().uuid() }))
    .query(async ({ input }) => {
      return prisma.pinnedMessage.findMany({
```

```
        where: { channelId: input.channelId },
        orderBy: { pinnedAt: "desc" },
      });
    }),

  // Bookmark a message
  bookmark: protectedProcedure
    .input(z.object({ messageId: z.string().uuid(), note: z.s
tring().optional() }))
    .mutation(async ({ input, ctx }) => {
      return prisma.messageBookmark.upsert({
        where: { userId_messageId: { userId: ctx.user.id, mes
sageId: input.messageId } },
        create: { userId: ctx.user.id, messageId: input.messa
geId, note: input.note },
        update: { note: input.note },
      });
    }),

  // Get user's bookmarks
  getBookmarks: protectedProcedure
    .input(z.object({ limit: z.number().default(50) }))
    .query(async ({ input, ctx }) => {
      return prisma.messageBookmark.findMany({
        where: { userId: ctx.user.id },
        orderBy: { createdAt: "desc" },
        take: input.limit,
      });
    }),

  // Forward a message to another channel
  forward: protectedProcedure
    .input(z.object({ messageId: z.string().uuid(), targetCha
nnelId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const original = await prisma.message.findUnique({ wher
```

```
e: { id: input.messageId } });
      if (!original) throw new Error("Message not found");

      return prisma.message.create({
        data: {
          channelId: input.targetChannelId,
          authorId: ctx.user.id,
          content: original.content,
          contentHtml: original.contentHtml,
          type: "user",
          metadata: { forwarded: true, originalMessageId: ori
ginal.id, originalAuthorId: original.authorId },
        },
      });
    }),
});
```

STEP 7: SOCKET CLIENT HOOK

Create file: `src/hooks/use-socket.ts`

```
// src/hooks/use-socket.ts

"use client";

import { useEffect, useRef, useCallback } from "react";
import { io, Socket } from "socket.io-client";
import { useMessagingStore } from "@/stores/messaging-store";

let globalSocket: Socket | null = null;

export function useSocket(workspaceId: string) {
  const socketRef = useRef<Socket | null>(null);
  const { addMessage, updateMessage, removeMessage, setTyping
Users, updatePresence, updateUnread } = useMessagingStore();
```

```
  useEffect(() => {
    if (globalSocket?.connected) {
      socketRef.current = globalSocket;
      return;
    }

    const socket = io(process.env.NEXT_PUBLIC_APP_URL || "",
{
      path: "/api/socket",
      transports: ["websocket", "polling"],
      withCredentials: true,
    });

    socket.on("connect", () => {
      console.log("Socket connected");
      socket.emit("join_workspace", { workspaceId });
    });

    // Message events
    socket.on("message_new", (data) => addMessage(data.channe
lId, data));
    socket.on("message_updated", (data) => updateMessage(dat
a.channelId, data));
    socket.on("message_deleted", (data) => removeMessage(dat
a.channelId, data.messageId));
    socket.on("reaction_updated", (data) => updateMessage(dat
a.channelId, data));

    // Typing
    socket.on("typing", (data) => setTypingUsers(data.channel
Id, data));

    // Presence
    socket.on("presence_update", (data) => updatePresence(dat
a.userId, data.status));
```

```
    // Unread
    socket.on("unread_updated", (data) => updateUnread(data.c
hannelId, data.unreadCount));

    socket.on("disconnect", () => console.log("Socket disconn
ected"));

    globalSocket = socket;
    socketRef.current = socket;

    return () => {
      // Don't disconnect on unmount — keep global connection
    };
  }, [workspaceId]);

  const sendMessage = useCallback((data: any) => {
    socketRef.current?.emit("message_send", data);
  }, []);

  const editMessage = useCallback((data: any) => {
    socketRef.current?.emit("message_edit", data);
  }, []);

  const deleteMessage = useCallback((data: any) => {
    socketRef.current?.emit("message_delete", data);
  }, []);

  const addReaction = useCallback((data: any) => {
    socketRef.current?.emit("reaction_add", data);
  }, []);

  const removeReaction = useCallback((data: any) => {
    socketRef.current?.emit("reaction_remove", data);
  }, []);
```

```
  const startTyping = useCallback((channelId: string) => {
    socketRef.current?.emit("typing_start", { channelId });
  }, []);

  const stopTyping = useCallback((channelId: string) => {
    socketRef.current?.emit("typing_stop", { channelId });
  }, []);

  const markRead = useCallback((channelId: string, messageId:
string) => {
    socketRef.current?.emit("mark_read", { channelId, message
Id });
  }, []);

  const setPresence = useCallback((status: string, customTex
t?: string) => {
    socketRef.current?.emit("presence_set", { status, customT
ext });
  }, []);

  return {
    socket: socketRef.current,
    sendMessage, editMessage, deleteMessage,
    addReaction, removeReaction,
    startTyping, stopTyping, markRead, setPresence,
  };
}
```

STEP 8: MESSAGING ZUSTAND STORE

Create file: `src/stores/messaging-store.ts`

```
// src/stores/messaging-store.ts

import { create } from "zustand";
```

```
interface MessagingState {
  // Active channel/conversation
  activeChannelId: string | null;
  activeConversationId: string | null;
  setActiveChannel: (id: string | null) => void;
  setActiveConversation: (id: string | null) => void;

  // Messages by channel
  messagesByChannel: Record<string, any[]>;
  addMessage: (channelId: string, message: any) => void;
  updateMessage: (channelId: string, message: any) => void;
  removeMessage: (channelId: string, messageId: string) => vo
id;
  setMessages: (channelId: string, messages: any[]) => void;

  // Typing users by channel
  typingUsers: Record<string, Array<{ userId: string; userNam
e: string }>>;
  setTypingUsers: (channelId: string, data: { userId: string;
userName?: string; isTyping: boolean }) => void;

  // Presence
  presenceMap: Record<string, string>; // userId → status
  updatePresence: (userId: string, status: string) => void;

  // Unread counts
  unreadCounts: Record<string, number>; // channelId → count
  updateUnread: (channelId: string, count: number) => void;

  // Thread panel
  activeThreadId: string | null;
  setActiveThread: (messageId: string | null) => void;
}

export const useMessagingStore = create<MessagingState>((set)
```

```
=> ({
  activeChannelId: null,
  activeConversationId: null,
  setActiveChannel: (id) => set({ activeChannelId: id, active
ConversationId: null }),
  setActiveConversation: (id) => set({ activeConversationId:
id, activeChannelId: null }),

  messagesByChannel: {},
  addMessage: (channelId, message) =>
    set((state) => ({
      messagesByChannel: {
        ...state.messagesByChannel,
        [channelId]: [...(state.messagesByChannel[channelId]
|| []), message],
      },
    })),
  updateMessage: (channelId, message) =>
    set((state) => ({
      messagesByChannel: {
        ...state.messagesByChannel,
        [channelId]: (state.messagesByChannel[channelId] ||
[]).map((m) =>
          m.id === message.messageId || m.id === message.id ?
{ ...m, ...message } : m
        ),
      },
    })),
  removeMessage: (channelId, messageId) =>
    set((state) => ({
      messagesByChannel: {
        ...state.messagesByChannel,
        [channelId]: (state.messagesByChannel[channelId] ||
[]).map((m) =>
          m.id === messageId ? { ...m, deletedAt: new Date().
toISOString() } : m
```

```
        ),
      },
    })),
  setMessages: (channelId, messages) =>
    set((state) => ({
      messagesByChannel: { ...state.messagesByChannel, [chann
elId]: messages },
    })),

  typingUsers: {},
  setTypingUsers: (channelId, data) =>
    set((state) => {
      const current = state.typingUsers[channelId] || [];
      if (data.isTyping) {
        if (!current.find((u) => u.userId === data.userId)) {
          return { typingUsers: { ...state.typingUsers, [chan
nelId]: [...current, { userId: data.userId, userName: data.us
erName || "" }] } };
        }
      } else {
        return { typingUsers: { ...state.typingUsers, [channe
lId]: current.filter((u) => u.userId !== data.userId) } };
      }
      return state;
    }),

  presenceMap: {},
  updatePresence: (userId, status) =>
    set((state) => ({ presenceMap: { ...state.presenceMap, [u
serId]: status } })),

  unreadCounts: {},
  updateUnread: (channelId, count) =>
    set((state) => ({ unreadCounts: { ...state.unreadCounts,
[channelId]: count } })),
```

```
  activeThreadId: null,
  setActiveThread: (messageId) => set({ activeThreadId: messa
geId }),
}));
```

EDGE CASES — SPRINT 1

```
1. Socket disconnects → 30s grace period before marking offli
ne (handles page refresh)
2. Send message to archived channel → reject with "This chann
el is archived"
3. Edit message after 15 min → reject with "Messages can only
be edited within 15 minutes"
4. Delete message in announcement channel by non-admin → reje
ct
5. React with same emoji twice → idempotent (upsert, no dupli
cate)
6. Typing indicator auto-expires after 5s (Redis TTL) if user
stops without explicit stop event
7. Unread count Redis key expires after 30 days to prevent me
mory leak
8. Message with content > 40,000 chars → reject with "Message
too long (max 40,000 characters)"
```

VERIFICATION — MESSAGING SPRINT 1

```
□ npx prisma migrate dev → all messaging tables created
□ Socket.IO server starts and accepts connections
□ Send message via socket → message saved in database + broad
cast to channel
□ Edit message → updated in database + broadcast
□ Delete message → soft deleted + broadcast "message_deleted"
□ Add reaction → saved + denormalized summary updated + broad
```

```
cast
□ Presence: connect → status "online" broadcast to workspace
□ Presence: disconnect → 30s delay → status "offline"
□ Typing: emit typing_start → others see typing indicator
□ Mark read → unread count cleared for that channel
□ tRPC messages.list → returns paginated messages for channel
□ Pin message → appears in getPinned query
□ Bookmark message → appears in getBookmarks query
```

Once all checks pass, proceed to Messaging Sprint 2.